COMPSYS302: Python

3. Functions and Arguments

Ho Seok AHN

hs.ahn@auckland.ac.nz

3.1 About Function

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

Functions

- Most statements in a typical Python program are organized into functions.
- A **function** is a group of statements that executes upon request.
- Python provides many built-in functions and allows programmers to define their own functions.
- **Function call**: a request to execute a function
- When a function is called, it may be passed arguments that specify data upon which the function performs its computation.
- In Python, **a function always returns a result value**, either none or value that represents the results of its computation.

3.2 Definition of Function

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

The def Statement

- The **def** statement is the most common way to define a function.
- **def** is a single-clause compound statement with the following syntax:

```
def function-name(parameters):
    statements
    return
```

  - **function-name** is an identifier.
  - **parameters** is an optional list of identifiers, that are used to represent values that are supplied as arguments when the function is called.
  - You should use indentation for statements.

3.3 Return Statement

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

☐ The return Statement

- The **return** statement in Python is allowed only inside a function body, and it can optionally be followed by an expression.
- When **return** executed, the function terminates, and the value of the expression is returned.
- A function returns none if it terminates by reaching the end of its body or by executing a return statement that has no expression.

  - Ex) 3_01_DefineFunction.py

```
def Times(a, b):
    return a*b
print Times(10, 20)
```

(result)

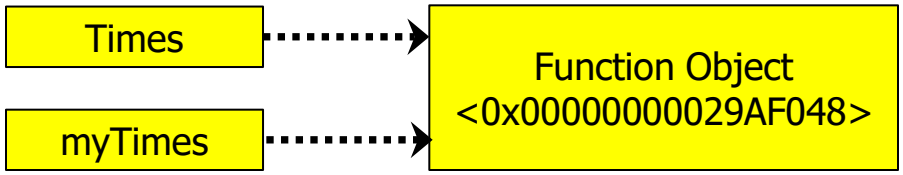
200

3.3 Return Statement

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

Memory Reference

- When a function is generated, memory address of the function object is assigned.
- And it is assigned differently every time.
- You can allocate the generated function to different variables.



- Ex) 3_02_MemoryReference.py

```
def Times(a, b):
    return a*b
print (Times(10, 20))
myTimes = Times
print (myTimes(10, 20))

(result)
200
200
```

3.3 Return Statement

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

Namespaces

- Local scope (local namespace)
  - An area for the function's formal parameters, plus any variables that are bound in the function body.
  - Each of these variables is called a local variable of the function.
  - Local variables are valid inside of function body.

- Global scope (global namespace)
  - An area for the parameters and variables that are not bound in the function body.
  - Each of these variables is called a global variable.
  - You can define a global variable inside of function body.

    (ex. 3_03_2_NameSpaces.py)

3.3 Return Statement

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

☐ Kinds of arguments

- Arguments that are just expressions are called as **positional arguments**.
- Each **positional argument** supplies the value for the formal parameter that corresponds to it by position in the function definition.
- In a function call, zero or more positional arguments may be followed by zero or more named arguments with the following syntax:

identifier = expression

- Ex) 3_04_ArgumentSetting.py

```
def Times(a=10, b=20):
    return a*b
print (Times())
print (Times(5))          # a = 5, b = 20
print (Times(b=30))       # a = 10, b = 30
print (Times(b=30, a=20))
```

(result)

```
200
100
300
600
```

3.4 lambda Expressions

THE UNIVERSITY OF AUCKLAND
FACULTY OF ENGINEERING

☐ lambda expression

- If a function body contains a single return expression statement, you may choose to replace the function with the special **lambda** expression form:

  lambda parameters: expression

- The **lambda** syntax does not use the return keyword.
- **lambda** can sometimes be handy when you want to use a simple function as an argument or return value.

  - Ex) 3_05_LambdaExpression.py

```
g = lambda a, b : a*b
print (g(10, 20))
```

(result)

200